Who Sees What — False Positives and Accepted Findings

Package: Who Sees What (`twinforce`) · Package Id `0HoOW00000000ZV0AY` **Version:** `[FILL IN]` **Scanner:** Salesforce Code Analyzer `[FILL IN VERSION, e.g. 0.52.0]` (PMD 7.26.0, RetireJS, Graph Engine) **Date:** `[FILL IN]`

Summary

The AppExchange and Security rulesets report **0 violations**:

```
sf code-analyzer run --workspace force-app --config-file code-
analyzer.yml \
  --rule-selector AppExchange --rule-selector Recommended:Security
→ Found 0 violations.
```

This document covers the six `@SuppressWarnings` annotations present in the source and two scanner conditions a reviewer may notice. **None of the suppressed rules appear in the AppExchange or Security rulesets** — verified by removing all six annotations and re-running the gate scan, which still returned 0 violations. They are retained so that the broader `Recommended` triage view stays quiet and genuine new findings are not lost among known ones.

Each is a design decision, documented in the code at the point of suppression as well as here.

---

1. `PMD.OperationWithLimitsInLoop` — 5 instances

Location: `PEAccessQueryService.cls`, on `applyTabVisibility`, `applyObjectPermissions`, `appendAccessWithoutTab`, `buildObjectGrants`, `buildFieldPerms`.

Why it fires: each method contains a SOQL query inside a `for` loop.

Why it is not a defect: the loop exists precisely to respect governor limits, not in ignorance of them.

A user's effective permission-set parents include their profile-as-permission-set, every directly assigned permission set, and every component of every assigned permission set group. In a large enterprise org this is routinely hundreds of Ids — more than a single SOQL bind list can safely carry. `PESoqlChunker.chunk()` splits the parent set into batches, and `PESoqlChunker.assertQueryBudget(batches.size())` checks the resulting query count against the governor limit **before any query runs**, throwing a typed `LIMIT_EXCEEDED` error the UI renders as a retryable per-section failure.

Hoisting the queries out of the loop is the bug this pattern prevents: a single query with an unbounded bind list either fails at runtime or silently truncates.

Security impact: none. This is a performance and correctness rule, not a security rule. It does not appear in the AppExchange or Security rulesets. Every query involved runs in user mode (`WITH USER_MODE`), so object and field permissions are enforced regardless of how the query is batched.

Mitigation in place: pre-flight query budget check; typed retryable error; Platform Cache on the resolved result so repeated drill-downs within a session do not re-run the chain.

---

2. `PMD.CognitiveComplexity` and `PMD.CyclomaticComplexity` — 1 class-level instance

Location: `PEAccessQueryService.cls`, class declaration.

Why it fires: class cognitive complexity 81 against a threshold of 50; cyclomatic complexity 51 against 50.

Why it is not a defect: the complexity is breadth, not depth. This class is the query surface for the whole product: six permission columns (create, read, update, delete, view all, modify all) across four distinct query shapes (tab visibility, object permissions, access-without-tab, field permissions). Individual method complexity was reduced during remediation by extracting `mergeCrud`, `permValues`, `apiNameByTabName`, `newAccessWithoutTabRow` and `seedFieldRows`; the class total remains above the threshold because the breadth is inherent.

Splitting the class to satisfy a numeric threshold would scatter the *absence rule* — a missing `ObjectPermissions` or `FieldPermissions` row means **no access**, never "unknown" — across multiple files. That rule is the one invariant that must hold identically in every code path in this product, and keeping it in one file is worth more than a metric.

Security impact: none. Complexity metrics are maintainability rules. Not in the AppExchange or Security rulesets.

---

3. `PMD.ApexSOQLInjection` — resolved, no longer suppressed

Included because a reviewer comparing scan runs may see this rule mentioned in earlier reports or in commit history.

Original condition: `PEUserSearchService.search` assembles one SOQL string at runtime to apply optional filters (term, profile, role, user type) and the keyset cursor predicate.

How it was resolved rather than suppressed:

- The query string is built exclusively from `private static final` constants declared at the top of the class. Filters are applied by **choosing which constant to append**, never by concatenating a value.
- All values reach the database through a named bind map passed to `Database.queryWithBinds(soql, binds, AccessLevel.USER_MODE)`.

- The only varying identifier is the sort column. It is not taken from input either: `sortField` is validated by `PEKeysetCursor.isAllowedSort`, falls back to `'Name'`, and the sort is then applied by selecting between two constant `ORDER BY` fragments. The cursor predicate is selected on the same field, so predicate and ordering cannot become mismatched.

Tests asserting this:

- `PEUserSearchServiceTest.quoteInTermIsBoundNotInterpolated` — a single quote in the search term is bound, not interpolated
- `PEUserSearchServiceTest.disallowedSortFieldFallsBackToName`
- `PermissionExplorerTest.disallowedSortFieldIsRejected`

The rule no longer fires and no suppression remains in the source.

---

4. Scanner conditions a reviewer may notice

4a. Graph Engine path-evaluation timeouts

Two entry points on `PermissionExplorerController` exceed the Graph Engine's default 30-second per-path budget:

```
Internal execution error while scanning entry point:
PermissionExplorerController.cls:<line>: Path evaluation timed out
after 30000 ms
```

These are `getObjectDetail` and `getAppObjects`, the two methods that fan out through the full resolver and query stack. The timeout is path explosion in the analyser, not a finding against the code — those entry points are reported as **not scanned**, not as passing.

Both are covered by the mitigations described in §4 of the architecture document: `PEAccessGate.assertAuthorised()` is the first statement in each, all classes are `with sharing`, and all SOQL runs in user mode. They are additionally exercised by `PermissionExplorerControllerTest`, `PermissionExplorerFlowTest`, and `PEUserDetailServiceTest`.

```
[IF YOU RAISED THE TIMEOUT AND THEY SCANNED CLEAN, REPLACE THIS
SECTION WITH THAT RESULT — a clean scan is a materially better answer
than this explanation.]
```

4b. PMD JavaScript rules are excluded from our configuration

`code-analyzer.yml` scopes PMD to Apex file extensions only:

```yaml
engines:
  pmd:
    file_extensions:
      apex:
```

```
    - .cls
    - .trigger
```

PMD's JavaScript ruleset predates ES modules and Lightning Web Components. Run against our LWC source it flagged 95 `GlobalVariable` violations, every one of which was a standard browser API — `window`, `document`, `URLSearchParams`, `setTimeout` — which PMD treats as an undeclared global. There were no true positives among them.

LWC linting is performed separately with ESLint and `@salesforce/eslint-config-lwc`, via `npm run lint`, which understands browser globals and LWC decorators natively.

The `code-analyzer.yml` file is included in the submitted package source for inspection.

---

5. Declarations

- **No callouts.** Searching the package source for `HttpRequest`, `new Http()`, and `callout:` returns zero matches. No Named Credentials, External Credentials, Remote Site Settings, or Auth Providers are present.
- **No credential storage.** No OAuth tokens, session tokens, API keys, or passwords are stored in any form.
- **No off-platform data storage or transmission.**
- **No `without sharing`.** All 16 non-test classes are `public with sharing`.
- **No `WITHOUT USER_MODE`** or `AccessLevel.SYSTEM_MODE` anywhere in the source.
- **No dynamic Apex.** No `Type.forName`, no `JSON.deserializeUntyped` on user-controlled input driving execution.